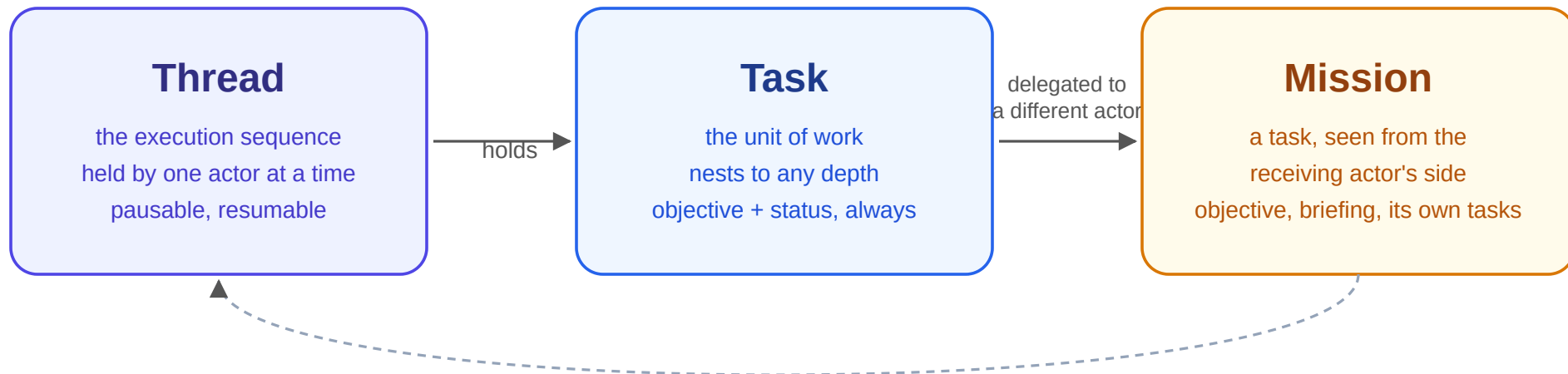




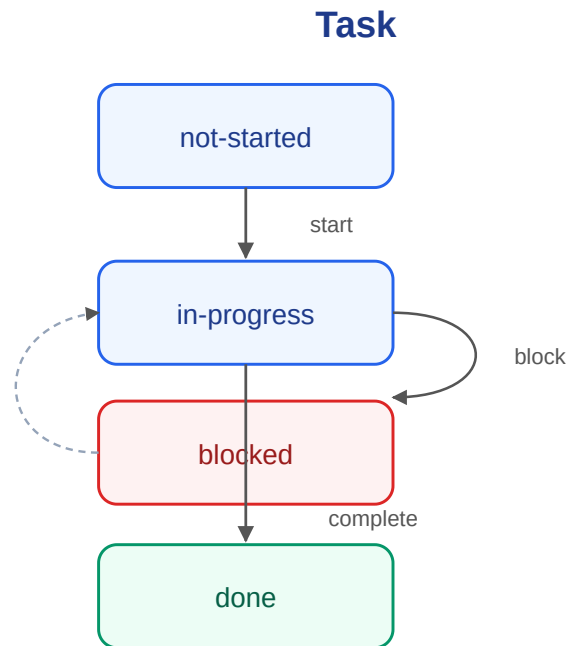
A working model for how software delivery gets tracked, for humans and agents together.

Three objects, not a generic ticket

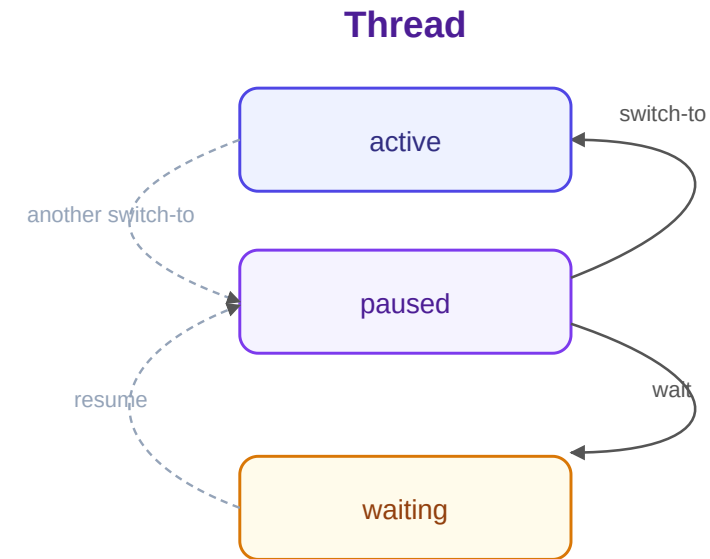
- **Thread**: the execution sequence. Held by one actor at a time.
- **Task**: the unit of work. Nests to any depth.
- **Mission**: a task, once delegated to a different actor.



Lifecycles are explicit, not implied by a label

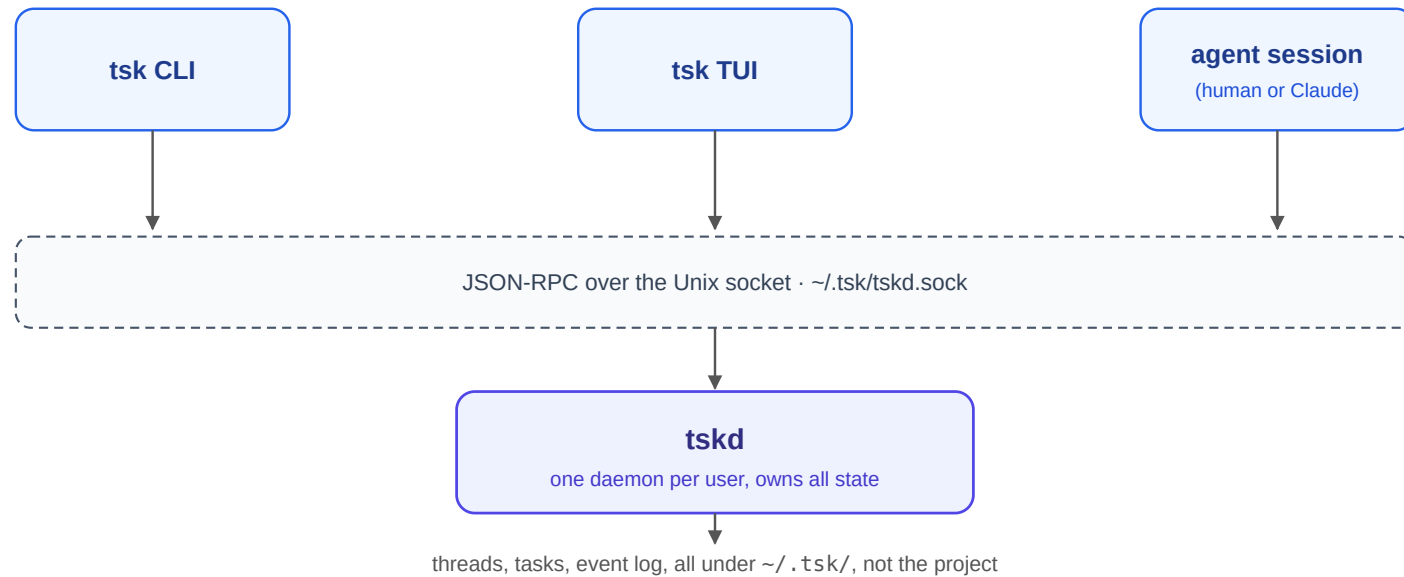


not-started / in-progress / blocked all cancel too (not drawn)



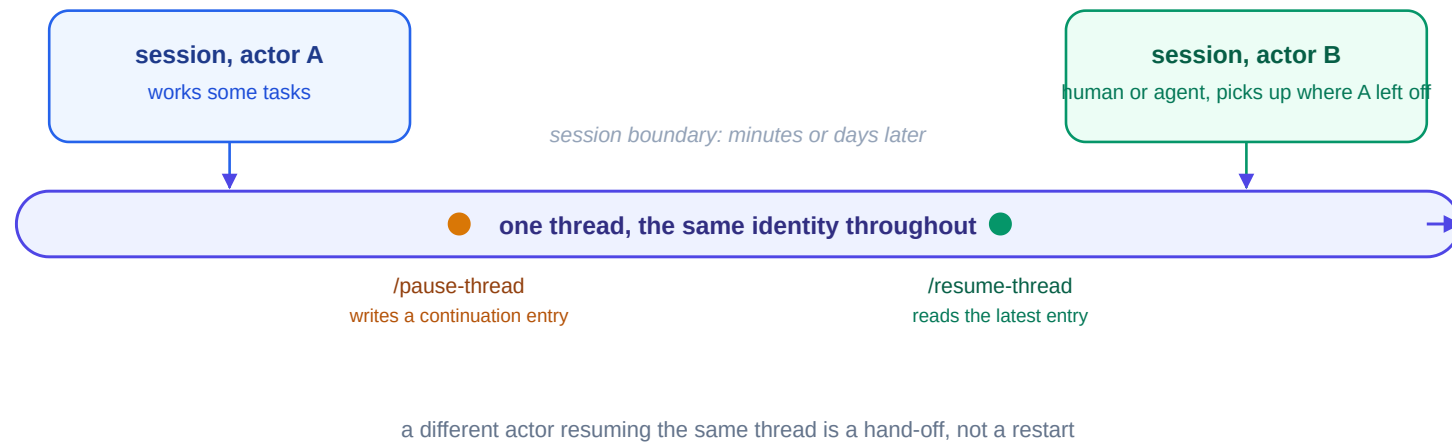
one actor holds a thread; a different actor resuming it is a hand-off, not a switch

One daemon, several clients



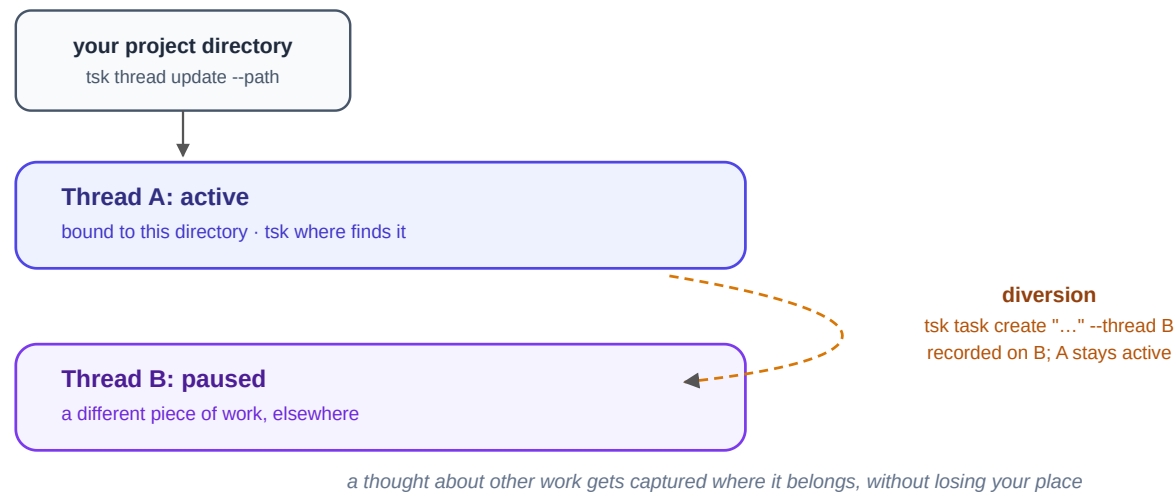
- **tskd** runs once per user and owns all state: an append-only event log plus a queryable cache.
- CLI, TUI and an agent session talk to it over the same JSON-RPC socket, concurrently, with state under `~/.tsk/`, outside any one project.

A thread survives the session that started it



- Pausing writes a continuation entry: the task in progress, and a short account of what to do next. Resuming reads it and reports where things stand, then asks before continuing.
- A different actor can pick up someone else's thread. That is a hand-off, recorded as such, not a silent takeover.

Fits how you actually move between things



- A thread can bind to a project directory: `tsk where` finds it, and `tsk` auto-zooms to it from inside that directory.
- Priorities (`BG` , `PRI0` , `INC`) triage which thread needs attention first.
- A diversion records a thought against a different thread without switching your active one.

What's built is Navigation

tsk today tracks threads, tasks and missions: the map and the route.

The wider bet is a model that also joins Delta, Product and Scale into the same structure.

[docs/vision.md](#)